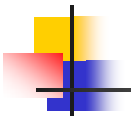
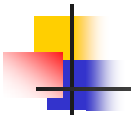


Programming Java

Exception Handling

Incheon Paik



Contents

Exception Handling

```
class DivideByZero {

  public static void main(String args[]) {
    a();
  }

  static void a() {
    b();
  }

  static void b() {
    c();
  }

  static void c() {
    d();
  }

  static void d() {
    int i = 1;
    int j = 0;
    System.out.println(i / j);
  }
}
```

Result :

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
        at DivideByZero.d(DivideByZero.java:22)
        at DivideByZero.c(DivideByZero.java:16)
        at DivideByZero.b(DivideByZero.java:12)
        at DivideByZero.a(DivideByZero.java:8)
        at DivideByZero.main(DivideByZero.java:4)
```


Exception Handling

Exception Handling

```
try {
  // try block
}
catch (ExceptionType1 param1) {
  // Exception Block
}
catch (ExceptionType2 param2) {
  // Exception Block
}
.....
catch (ExceptionTypeN paramN) {
  // Exception Block
}
finally {
  // finally Block
}
```

Statements that have some possibilities to generate exception(s).

Execute statements here when the corresponding exception occurred.

Do always

Exception Handling

```
class Divider {

  public static void main(String args[]) {

    try {

      System.out.println("Before Division");
      int i = Integer.parseInt(args[0]);
      int j = Integer.parseInt(args[1]);
      System.out.println(i / j);
      System.out.println("After Division");
    }
    catch (ArithmeticException e) {
      System.out.println("ArithmeticException");
    }
    catch (ArrayIndexOutOfBoundsException e) {
      System.out.println("ArrayIndex" +
        "OutOfBoundsException");
    }
    catch (NumberFormatException e) {
      System.out.println("NumberFormatException");
    }
```

```
    finally {
        System.out.println("Finally block");
      }
    }
}
```

Result : java Divider
Before Division
ArrayIndexOutOfBoundsException
Finally block

Result : java Divider 1 0
Before Division
ArithmeticException
Finally block

Catch Block Searches

```
class CatchSearch {

  public static void main(String args[])
  {
    try {
      System.out.println("Before a");
      a();
      System.out.println("After a");
    }
    catch (Exception e) {
      System.out.println("main: " + e);
    }
    finally {
      System.out.println("main:
finally");
    }
  }

  public static void a() {
    try {
      System.out.println("Before b");
      b();
      System.out.println("After b");
    }
    catch (ArithmeticException e) {
      System.out.println("a: " + e);
    }
    finally {
      System.out.println("a: finally");
    }
  }
```

```
  public static void b() {
    try {
      System.out.println("Before c");
      c();
      System.out.println("After c");
    }
    catch
(ArrayIndexOutOfBoundsException e)
    {
      System.out.println("b: " + e);
    }
    finally {
      System.out.println("b: finally");
    }
  }

  public static void c() {
    try {
      System.out.println("Before d");
      d();
      System.out.println("After d");
    }
    catch (NumberFormatException e) {
      System.out.println("c: " + e);
    }
    finally {
      System.out.println("c: finally");
    }
  }
```

```
  public static void d() {
    try {
      int array[] = new int[4];
      array[10] = 10;
    }
    catch (ClassCastException e) {
      System.out.println("d: " + e);
    }
    finally {
      System.out.println("d: finally");
    }
  }
}
```

Result :
Before a
Before b
Before c
Before d
d: finally
c: finally
b: java.lang.ArrayIndexOutOfBoundsException: 10
b: finally
After b
a: finally
After a
main: finally

The **throw** Statement

The throw Statement

throw object;

Use the statement to generate an exc eption

Exception Object in Argument

```
catch (ExceptionType param) {
 ....
  throw param;
 .....
}
```

Throw to new Exception Object

throw new ExceptionType(args);

The **throw** Statement

```
class ThrowDemo {

 public static void main(String args[])
{
    try {
      System.out.println("Before a");
      a();
      System.out.println("After a");
    }
    catch (ArithmeticException e) {
      System.out.println("main: " + e);
    }
    finally {
      System.out.println("main:
finally");
    }
  }

  public static void a() {
    try {
      System.out.println("Before b");
      b();
      System.out.println("After b");
    }
    catch (ArithmeticException e) {
      System.out.println("a: " + e);
    }
    finally {
      System.out.println("a: finally");
    }
```

```
  public static void b() {
    try {
      System.out.println("Before c");
      c();
      System.out.println("After c");
    }
    catch (ArithmeticException e) {
      System.out.println("b: " + e);
    }
    finally {
      System.out.println("b: finally");
    }
  }

  public static void c() {
    try {
      System.out.println("Before d");
      d();
      System.out.println("After d");
    }
    catch (ArithmeticException e) {
      System.out.println("c: " + e);
      throw e;
    }
    finally {
      System.out.println("c: finally");
    }
  }
```

```
  public static void d() {
    try {
      int i = 1;
      int j = 0;
      System.out.println("Before division");
      System.out.println(i / j);
      System.out.println("After division");
    }
    catch (ArithmeticException e) {
      System.out.println("d: " + e);
      throw e;
    }
    finally {
      System.out.println("d: finally");
    }
  }
}
```

Result :

```
Before a
Before b
Before c
Before d
Before division
d: java.lang.ArithmeticException: / by zero
d: finally
c: java.lang.ArithmeticException: / by zero
c: finally
b: java.lang.ArithmeticException: / by zero
b: finally
After b
a: finally
After a
main: finally
```

The **throw** Statement

```
class ThrowDemo2 {

  public static void main(String args[]) {
    try {
      System.out.println("Before a");
      a();
      System.out.println("After a");
    }
    catch (Exception e) {
      System.out.println("main: " + e);
    }
    finally {
      System.out.println("main: finally");
    }
  }
```

```
  public static void a() {
    try {
      System.out.println("Before throw statement");
      throw new ArithmeticException();
    }
    catch (Exception e) {
      System.out.println("a: " + e);
    }
    finally {
      System.out.println("a: finally");
    }
  }
}
```

Result :
Before a
Before throw statement
a: java.lang.ArithmeticException
a: finally
After a
main: finally

Exception and Error Classes

Throwable Constructors

Throwable()
Throwable(String message)

getMessage() Method

String getMessage()

printStackTrace() Method

void printStackTrace()

Exception Constructor

Exception()
Exception(String message)

Subclasses of Exception Class

ClassNotFoundException
IllegalAccessException
InstantiationException
InterruptedException
NoSuchFieldException
NoSuchMethodException
RuntimeException

Subclasses of RuntimeException

ArrayIndexOutOfBoundsException
ArithmeticException
ClassCastException
NegativeArraySizeException
NullPointerException
NumberFromatException
SecurityException
StringIndexOutOfBoundsException

Exception and Error Classes

```
class PrintStackTraceDemo {

  public static void main(String args[]) {
    try {
      a();
    }
    catch (ArithmeticException e) {
      e.printStackTrace();
    }
  }

  public static void a() {
    try {
      b();
    }
    catch (NullPointerException e) {
      e.printStackTrace();
    }
  }

  public static void b() {
    try {
      c();
    }
    catch (NullPointerException e) {
      e.printStackTrace();
    }
  }

  public static void c() {
    try {
      d();
    }
    catch (NullPointerException e) {
      e.printStackTrace();
    }
  }

  public static void d() {
    try {
      int i = 1;
      int j = 0;
      System.out.println(i / j);
    }
    catch (NullPointerException e) {
      e.printStackTrace();
    }
  }
}
```

Result:

```
java.lang.ArithmeticException: / by zero
        at PrintStackTraceDemo.d(PrintStackTraceDemo.java:43)
        at PrintStackTraceDemo.c(PrintStackTraceDemo.java:32)
        at PrintStackTraceDemo.b(PrintStackTraceDemo.java:23)
        at PrintStackTraceDemo.a(PrintStackTraceDemo.java:14)
        at PrintStackTraceDemo.main(PrintStackTraceDemo.java:5)
```


The **throws** Clause

throws Statement of Constructor

```
consModifiers clsName(cparams) throws exceptions {
  // constructor body
}
```

throws Statement at a Method

```
mthModifiers rtype mthName(mparams) throws exceptions {
 // constructor body
}
```

```
class ThrowsDemo {

  public static void main(String args[]) {
    a();
  }

  public static void a() {
    try {
      b();
    }
    catch (ClassNotFoundException e) {
      e.printStackTrace();
    }
  }

  public static void b() throws ClassNotFoundException {
    c();
  }

  public static void c() throws ClassNotFoundException {
    Class cls = Class.forName("java.lang.Integer");
    System.out.println(cls.getName());
    System.out.println(cls.isInterface());
  }
}
```

Result :

```
Java.lang.Integer
false
```

Custom Exceptions

Subclass of Exception

```
import java.util.*;

class ExceptionSubclass {

  public static void main(String args[]) {
    a();
  }

  static void a() {
    try {
      b();
    }
    catch (Exception e) {
      e.printStackTrace();
    }
  }

  static void b() throws ExceptionA {
    try {
      c();
    }
    catch (ExceptionB e) {
      e.printStackTrace();
    }
  }
```

```
  static void c() throws ExceptionA, ExceptionB {
    Random random = new Random();
    int i = random.nextInt();
    if (i % 2 == 0) {
      throw new ExceptionA("We have a problem");
    }
    else {
      throw new ExceptionB("We have a big problem");
    }
  }
}

class ExceptionA extends Exception {
  public ExceptionA(String message) {
    super(message);
  }
}


class ExceptionB extends Exception {
  public ExceptionB(String message) {
    super(message);
  }
}
```

Execution:

```
ExceptionA: We have a problem
        at ExceptionSubclass.c(ExceptionSubclass.java:31)
        at ExceptionSubclass.b(ExceptionSubclass.java:20)
        at ExceptionSubclass.a(ExceptionSubclass.java:11)
        at ExceptionSubclass.main(ExceptionSubclass.java:6)
```


Exercise

Step 1 (Creating Some Methods for Exception)

Slides 7,8,9, 12

```
static int thrower(String s) ..... {
        try {
            if (s.equals("divide")) {
                // Write the code for raising Divide by Zero Exception
            }
            if (s.equals("null")) {
                // Write the code for raising NullPoint Exception
            }
            if (s.equals("test"))
                // Write the code for raising TestException
            return 0;
        } finally {
                // Some code for print out the current message
            }
```

Exercise

Step 2, 3 Writing a Exception Handler 1,2

Slides # 7,8,9, 12,13